

sudo

Tech, distilled.

The complete engineering guide to the app: what it does, how every part works, which technologies were used and why, how the on-device ML model was trained, how the Skills Radar verifies new terms, how the app is tested for truthfulness, and the questions you are most likely to be asked about it — with answers.

Android · Kotlin · Jetpack Compose · On-device ML

github.com/JiviteshKumar/sudo · October 2026

Contents

1 The 60-second pitch

2 What the app does

3 Design principles

4 Tech stack — and why each choice

5 Architecture

6 Data model

7 The news pipeline

8 The jobs pipeline

9 Trending now & Rising on GitHub

10 Skills Radar (term discovery & verification)

11 The on-device summarizer (the ML model)

12 Where LLMs are used (Gemini) — and how they're kept honest

13 Background work, performance & memory

14 Trust & verification

15 Testing strategy

16 Security & privacy

17 Build, release & GitHub

18 Key decisions & trade-offs

19 Bugs found and how they were fixed

20 Limitations & roadmap

21 Interview questions — with answers

22 Glossary

> **How to use this guide**
>
> Read sections 1–3 to explain the product, 4–5 to explain the engineering, and 10–11 for the ML depth an AI engineer will be probed on. Section 19 is your best "tell me about a hard problem" material. Section 21 is a rehearsal sheet. Every number in this guide comes from the code, the training report (`ml/REPORT.md`) or the live audit (`testing/TRUTH_REPORT.md`).

1 The 60-second pitch

sudo is an Android app that keeps you current in tech and AI. It pulls news only from established outlets and official AI-lab blogs, summarises each story in about 50 words *on the phone* with a small model I trained, tracks new and rising AI/ML/data-science terms (the "Skills Radar"), and lists jobs and internships matched against your resume — with a one-tap assisted apply and an application tracker.

What makes it different

- **It can't make things up.** Summaries are the publisher's own sentences, chosen by an on-device model — nothing is generated. Every card links to its source.
- **Nothing is hardcoded.** The Skills Radar mines new terms from live research and verifies each one against independent sources (OpenAlex publication history + arXiv activity) before showing it.
- **It proves it.** A live "truth audit" test re-checks headlines, links, summaries, job pages, trending items and radar terms against the original sources.
- **No backend.** Everything runs on the phone; there is no server to run or pay for.

Numbers worth remembering

What	Value
Code	~9,000 lines of Kotlin (app) + ~2,000 lines of tests, 43 source files (v1.2.0)
News sources	25 curated feeds (10 tech outlets, 15 AI/ML/DS sources incl. arXiv)
Job sources	5 job APIs (Adzuna needs a free key) + 18 company career boards (Greenhouse, Lever, Ashby); 1,782 roles in the last audit
Summarizer model	20 features → 24 hidden units → 1 output = **529 parameters**, 12 KB file
Training data	4,924 articles after filtering (CNN/DailyMail + articles from the app's own feeds); 3,909 used for training = 115,957 sentences
Summary quality	Beats the "first sentences" baseline: ROUGE-1 28.4 vs 27.4 (tech), 33.0 vs 31.6 (CNN)
APK size	~3.9 MB (R8-minified release)
Responsiveness	992 swipes during a live refresh: median 9 ms, p95 25 ms, max 501 ms (ANR limit 5,000 ms)
Memory fix	Radar mining went from +142 MB to +77 MB while handling twice the data
Audit result (6 Oct 2026)	80/80 links on publisher sites, 80/80 headlines match the feed, 15/15 checked summaries verbatim, 45/45 trending items exact

2 What the app does

⚡ Tech Pulse

Tech news from The Verge, Ars Technica, TechCrunch, Wired, Engadget, BBC, The Register, MIT Technology Review, Rest of World and The Hacker News. Topic chips (AI, Security, Startups, Gadgets…), a hero story, ~50-word summaries, "+N outlets" when several outlets cover the same story.

✦ AI & Data

Official lab blogs (Google AI, Google Research, DeepMind, OpenAI, Hugging Face, NVIDIA, AWS ML, MIT News AI), AI desks (Ars AI, TechCrunch AI, The Decoder), data-science (KDnuggets, Towards Data Science) and arXiv cs.LG / cs.CL. Plus Skills Radar and Trending now.

◎ Skills Radar

A permanent, month-by-month record of new and rising AI/ML/DS terms and tools. Each has a quoted "what it is", "where it's used", the numbers behind it and its sources.

▤ Trending now

Hugging Face trending models, Hugging Face Daily Papers (by upvotes) and rising AI/ML repos on GitHub — straight from their live sources.

🚀 Rising on GitHub

The 15 repos gaining stars fastest, with a direct link, the author's description and a quoted README paragraph. All / AI/ML / For you filters; updates every 3rd morning and keeps the list unchanged when nothing new makes it.

💼 Jobs & Internships

Remote and on-site roles; filters for internships, full-time, remote, on-site, near me and "direct from employer"; a match score and skill gaps computed against your resume; a scam filter.

🚀 Assisted apply + Tracker

Tap-to-copy details, an optional tailored cover letter, opens the employer's real form; you submit. Tracker: Saved → Applied → Interview → Offer / Rejected with a 7-day follow-up reminder.

🔔 Alerts & widget

Notifications when a headline or new job mentions your keywords; a home-screen headlines widget (Glance).

🔖 Saved & offline

Bookmarked stories are stored locally and never expire. Launch intro (~2.4 s), light/dark themes, onboarding.

Why ”assisted apply” instead of auto-apply

The original idea was an app that fills and submits applications automatically. That was rejected on purpose: LinkedIn/Indeed terms forbid automated applications (accounts get banned), every employer uses a different system (Workday, Greenhouse, Lever…) with CAPTCHAs and custom questions — so a bot would fail silently and often. Assisted apply keeps the speed (everything ready to paste) while the user stays in control and nothing is sent without review.

3 Design principles

1. **Trust first.** Only established sources. Every item shows where it came from and links to the original. Undated, future-dated, off-site and promotional items are rejected (Section 14).
2. **Extract, don't generate.** The default summaries and all radar explanations are verbatim quotes. Generated text (Gemini) is optional, labelled, and must pass a grounding check that rejects any number, name or claim not present in the sources.
3. **No hardcoded content.** Terms, trends and acronym meanings are learned from live data. What is fixed is the *list of trusted outlets* — deliberately, because it's the trust anchor.
4. **Never guess.** If a verification source is unreachable or rate-limited, the radar pauses and says so instead of showing unverified results.
5. **On-device and private.** No server, no analytics, no account. Keys and your resume stay on the phone.
6. **Honest labels.** "AI summary" vs "Key sentences · on-device" vs "Excerpt"; "live on careers page" instead of "posted 31 months ago"; "took off in 2024".
7. **Prove it with tests.** A live audit checks the app's output against reality, not just that the code runs.

4 Tech stack — and why each choice

Layer	Choice (version)	Why
Language	**Kotlin** 2.1.0, JVM target 17	Google's preferred Android language; null-safety; coroutines for async; the same code runs in JVM unit tests.
UI	**Jetpack Compose** (BOM 2024.12.01), Material 3	Declarative UI, easy animation (intro, gradients, chips), state-driven lists; the modern Android standard.
Navigation	Navigation-Compose 2.8.5	Single-activity app with type-safe-ish routes (home, job/{id}, radar, profile, settings, onboarding).
State	ViewModel + StateFlow (Lifecycle 2.8.7)	Unidirectional data flow: DB → Flow → ViewModel → UI. Survives rotation.
Database	**Room** 2.6.1 (code generated by KSP 2.1.0-1.0.29)	SQLite with compile-time-checked queries and reactive Flows; offline reading, tracker, radar history.
Settings	DataStore Preferences 1.1.1	Async, transactional replacement for SharedPreferences.
Background	**WorkManager** 2.10.0	The only reliable way to run periodic work on Android (survives reboots, respects Doze). 15 min is Android's minimum period.
Networking	OkHttp 4.12.0	Robust HTTP client: timeouts, redirects, gzip, streaming large responses.
JSON	kotlinx.serialization 1.7.3	Kotlin-native, reflection-free (works with R8); streaming decode for 10–15 MB job-board responses.
HTML / RSS	Jsoup 1.18.3 + XmlPullParser	Jsoup cleans HTML and extracts article text; a custom forgiving RSS/Atom pull parser handles broken feeds.
Images	Coil 2.7.0	Compose-native image loading with memory (capped at 12%) and disk caches.
Widget	Glance 1.1.1	Home-screen widget written in Compose style.
Browser	androidx.browser (Custom Tabs)	Opens articles and job forms in a fast in-app Chrome tab with the user's logins.
Concurrency	Kotlin coroutines 1.9.0	Structured concurrency; parallel fetches with semaphores; a dedicated background-priority dispatcher.
Build	Gradle 8.11.1, AGP 8.7.3, R8	compileSdk/targetSdk 35, minSdk 26 (Android 8.0+). R8 shrinks the release APK to ~3.9 MB.
Tests	JUnit 4, Robolectric 4.14.1, Compose UI test, Roborazzi 1.32.2, work-testing	Run Android/Compose code on the PC (no emulator needed), including screenshots.
Optional AI	Gemini REST API (default `gemini-2.5-flash-lite`)	Free tier; used only if the user adds a key. Model auto-discovered if the default is retired.

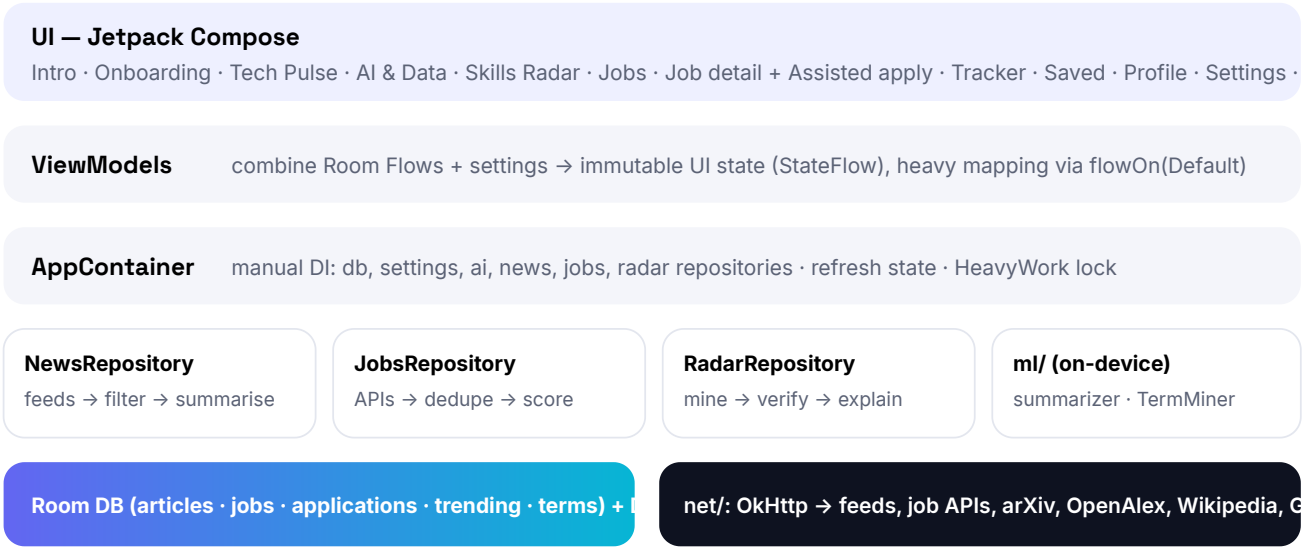
Why native Kotlin and not Flutter / React Native?

The app relies heavily on Android-specific capabilities: WorkManager background refresh, battery-optimisation exemption, notifications, a home-screen widget, Custom Tabs and a file provider for the resume. Native Kotlin gives first-class access to all of them, the best performance for long scrolling lists, and the ability to run the exact same parsing/ML code in fast JVM unit tests. The trade-off is no iOS version.

Why no backend?

A server would have to fetch, store and serve content — cost, maintenance and a privacy surface. Every source used is public and free, so the phone can fetch directly. The cost of this choice is that each phone does its own work (handled by the background worker, memory budget and rate-limit handling described later).

5 Architecture



Layered, single-activity architecture with unidirectional data flow. WorkManager calls the same repositories every 15 minutes.

Key patterns

- **Repository pattern:** each feature's data logic (fetch, clean, verify, store) lives in a repository; the UI only observes the database.
- **Single source of truth:** network results are written to Room; screens observe Room Flows. That's why the app shows cached data instantly and works offline.
- **Manual dependency injection:** one `AppContainer` object builds everything. For an app this size, Hilt/Koin would add complexity without benefit; tests pass an in-memory DB into the container.
- **Pure, testable core:** `NewsFilter`, `RssParser`, `TermMiner`, `ExtractiveSummarizer`, `Skills` have no Android dependencies, so they are unit-tested on the JVM.

Code map

Package	Files (lines)	Responsibility
`data/`	Database (333), RadarRepository (388), NewsRepository (186), JobsRepository (142), AiFeatures (196), Settings (180), HeavyWork (27)	Entities, DAOs, repositories, settings, Gemini prompts
`ml/`	TermMiner (365), Features (125), ExtractiveSummarizer (110), SentenceSplitter (52)	On-device ML: summariser and term mining
`net/`	JobSources (368), ResearchSources (293), RssParser (125), Gemini (109), Feeds (75), Http (70), NewsFilter (60), ArticleFetcher (27)	All network sources and parsing
`ui/`	NewsScreen (671), JobDetailScreen (611), RadarScreen (409), SettingsScreen (362), JobsScreen (360), Components (359), AppRoot (303), ProfileScreen (272), TrackerScreen (248), OnboardingScreen (224), IntroScreen (162), Theme (154)	Compose screens, design system
`work/`, `widget/`, `util/`	Notifications (132), Workers (76), HeadlinesWidget (98), Skills (142), Text (128), Browser (43), Battery (27)	Background jobs, widget, helpers

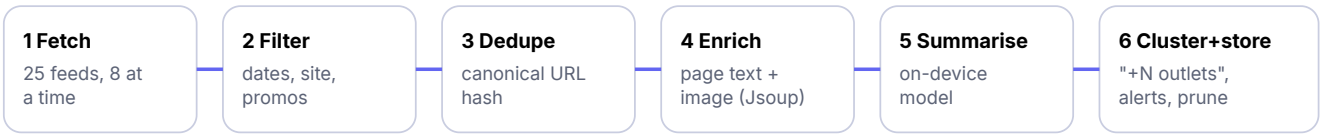
6 Data model

Table	Key fields	Notes
`articles`	id (hash of section + canonical URL), url, title, source, section, topic, imageUrl, publishedAt, excerpt, summary, summaryKind (AI / ON_DEVICE / EXCERPT), whyItMatters, clusterId, bookmarked, read	Pruned after 7 days unless bookmarked
`jobs`	id (source:externalId), title, company, location, isRemote, employmentType, isInternship, url, source, directFromEmployer, description (≤12k chars), postedAt, fetchedAt, salary, matchScore, matchedSkills, missingSkills, scamFlags, aiAnalysis	Lists use a light projection (no description) for speed
`applications`	jobId, title, company, url, status, appliedAt, followUpAt, notes, coverLetter	Kept separately so pruning jobs never deletes your tracker
`trending`	kind (models / papers), id, rank, title, subtitle, url, metric, fetchedAt	Refreshed hourly
`rising_repos`	fullName, url, description, readme (quoted intro), language, topics, stars, forks, createdAt, pushedAt, weekStars, prevStars + prevAt (previous check), growthPerDay + label + source, isAi, rankAll, rankAi, firstFeaturedAt, checkedAt	Added in database v2 by a migration; unlisted repos kept 30 days

`terms`	key, term, shortForm, kind (Concept / Tool), status (New / Rising / Rejected), reason, what + source + URL, usage + source + URL, simpleWhat, papers30, papersPrior, firstSeen, newsMentions, jobMentions, jobCompanies, evidence (JSON), score, checkedAt, discoveredAt	Verified terms kept permanently; rejected ones forgotten after 30 days

Settings (DataStore) hold API keys, location, keywords, theme, toggles, last-refresh timestamps and the radar rules version. The resume PDF is stored in app-private storage (`files/resume/`).

7 The news pipeline



1. **Fetch.** All 25 feeds are downloaded in parallel (a semaphore limits it to 8 at once). A failing feed never blocks the others.
2. **Filter** (`NewsFilter.reject`): an item must have a real publish date, no more than 2 hours in the future (clock-skew tolerance) and not older than 72 hours; must link to the *same publisher* as the feed (compared by "brand", e.g. `bbc` for bbc.com and bbc.co.uk, also accepting the site the feed declares in its own `<link>`); must be http(s); must have a headline; must not be commerce (promo codes, coupons, deals, ticket and event promos).
3. **Dedupe.** ID = a SHA-1 hash (first 20 hex characters) of section + canonical URL (tracking parameters like `utm_` removed). The same story can exist in both Tech and AI tabs.
4. **Enrich.** Of the 80 newest new items, the first 40 whose feed text is short (<400 chars) or that have no image get their article page fetched (6 at a time; arXiv is skipped) and Jsoup extracts the body paragraphs and the `og:image` .
5. **Summarise.** The on-device extractive model picks the key sentences (Section 11). If the text is too thin, the publisher's excerpt is shown, labelled "Excerpt". If the user enabled Gemini, up to 20 recent items per refresh are rewritten in batches of 10 and labelled "AI summary".
6. **Cluster.** Headlines are tokenised; two stories in the same section within 48 h are the same story if Jaccard ≥ 0.45, or ≥4 shared words with ≥60% overlap. arXiv papers only merge on identical titles (different papers often share vocabulary).
7. **Store & alert.** Inserted into Room; keyword alerts sent (never on the very first sync); stories older than 7 days are pruned unless bookmarked.

Topics (AI, Security, Startups, …) come from keyword rules, optionally overridden by Gemini. **Clustering** is a trust feature: "covered by 4 outlets" is a signal that a story is real.

8 The jobs pipeline

Source	Type	Notes
Remotive	Job site API	Remote; software-dev, data and devops categories
Remote OK	Job site API	Remote; like every source, its name is shown on each card ("via Remote OK")
The Muse	Job site API	Software engineering, data science, internships
Arbeitnow	Job site API	Europe-heavy
Adzuna	Job site API (optional free key)	Best for on-site jobs near you; 19 countries including India

Greenhouse, Lever, Ashby	Company career boards	Direct from employer: Anthropic, Databricks, Stripe, Figma, Airbnb, Discord, Cloudflare, Scale AI, GitLab, MongoDB, Dropbox, Pinterest, Palantir, Spotify, OpenAI, Notion, Linear, Ramp (editable)

- **Tech-role filter:** "strong" title words (engineer, developer, scientist, data, ML…) always qualify; "weak" ones (AI, research, analyst…) only if the title isn't clearly non-tech (sales, recruiter, marketing…).
- **Dedupe:** same title + company + location across sources → the employer's own posting wins.
- **Freshness:** job-site listings older than 45 days are dropped; employer-board listings are kept while the company still lists them (labelled "live on careers page"); at most 120 tech roles are taken per company board. Listings not seen at their source for 3 days are deleted.
- **Large responses:** some boards return 10–15 MB of JSON (OpenAI's ~14.6 MB). They are decoded straight from the network stream into small typed models (kotlinx.serialization `decodeFromStream`) instead of building a full JSON tree, to stay within phone memory.
- **Sync cadence:** at most hourly (the 15-minute worker skips jobs if the last sync was <55 min ago); pull-to-refresh forces it.

Match score and skill gaps

A fixed vocabulary of 82 canonical skills with aliases (e.g. *k8s → Kubernetes*, *sklearn → scikit-learn*) is matched with word-boundary regexes against the job text and the user's skills. **Score = matched ÷ min(required, 10) × 100.** The cap of 10 stops long "nice to have" lists from punishing everyone. It's deterministic and explainable — you can always see which skills matched and which are missing. Missing skills link to a YouTube course search. An optional Gemini "deep analysis" gives a fit score, strengths, gaps and eligibility notes from the full resume.

Scam filter

Regex red flags: asks for a fee or deposit, contact via WhatsApp/Telegram, "no interview", too-good pay claims, asks for ID/bank details, CV to a personal Gmail. One flag shows a warning; two or more hide the listing.

Assisted apply

The sheet has four steps: copy your details (name, email, phone, links); generate a cover letter and three answers (Gemini, instructed to use only resume facts); resume reminder and share button; open the employer's page. When you return, the app asks "Did you submit it?" — yes moves it to Applied and schedules a 7-day follow-up reminder through WorkManager.

9 Trending now & Rising on GitHub

Trending now (AI & Data tab)

Tab	Live source	Ranking
Models	`huggingface.co/api/models?sort=trendingScore`	Hugging Face's own trending score
Papers	`huggingface.co/api/daily_papers`	Upvotes
Repos	The AI/ML list of Rising on GitHub (below)	Stars gained recently

Nothing is hardcoded: the Hugging Face lists are fetched hourly and each tab's caption states its source and ranking. The Repos tab used to list new AI repos by total stars; it was replaced by Rising on GitHub's AI/ML list, which also catches older repos that suddenly take off.

Rising on GitHub (Tech Pulse tab, added in v1.2.0)

The 15 repos gaining stars fastest right now, with a direct link, the author's own description ("What it is") and the first paragraph of the README, quoted ("What it does"). Filters: **All**, **AI/ML** and **For you**.

Step	How it works
Discover	GitHub's Trending page (weekly), which shows each repo's "N stars this week", plus four GitHub searches for repos launched in the last 45 days (all of GitHub, and the `llm`, `ai-agents` and `machine-learning` topics). GitHub has no official API for Trending, so the page is parsed; if it changes, the searches still work.
Confirm	Every Trending repo is looked up in GitHub's public API (no key needed). Left out: archived repos, forks, repos with no description, repos not pushed to in 30 days.
Measure growth	Stars per day from the strongest evidence: Trending's weekly count; otherwise the gain sudo measured between two of its own checks (≥20 h apart); otherwise, for a repo under 45 days old, its average since launch. Every card says which source its number came from. A repo with no evidence of growth is never listed.
Keep or renew	The top 15 by growth replace the list only if that is a different set of repos. Same repos → the list stays exactly as it was (no reshuffling). New entries get a "New" badge.
Schedule	Every 3rd morning: due 3 days after the last scheduled update (minus 4 h slack) between 6 am and noon local time, or after 4 days at any time if every morning was missed. The 15-minute worker checks whether it's due; opening Tech Pulse does too. A manual "Check now" is allowed once an hour and doesn't move the schedule.
AI/ML filter	The repo's own topics (e.g. `llm`, `ai-agents`, `pytorch`) or its own description (AI, LLM, agents, Claude, diffusion…; "user agent" is excluded).
For you	Repos whose name, description, README quote, language or topics mention a skill from your profile, using the same 82-skill matcher as jobs.
Budget	GitHub allows 60 API requests an hour per network without a token. A first update used 14 (measured on 20 Oct 2026); later ones also re-check listed repos, so up to about 40. README text comes from `raw.githubusercontent.com`, which isn't counted. If the limit or an outage is hit, the list is kept unchanged and checked again later; nothing is guessed.

A trust decision worth knowing: no ”fake stars” flag

Star counts can be bought, so I planned to flag repos with many stars but few watchers. Before building it I measured 12 of the most-starred repos created in the past year: every one, including well-known legitimate projects, had watchers at only 0.2–1% of stars. The signal can't tell viral from fake, so the app shows the facts instead (stars, forks, created date, last update) plus a note that stars measure popularity, not quality or safety.

Data is stored in the `rising_repos` table (added by a database migration from version 1 to 2, so updating the app keeps bookmarks, the tracker and radar history). Repos that leave the list are kept for 30 days so their

growth can be measured if they come back.

10 Skills Radar — term discovery & verification

The most "AI-engineer" part of the app: an automatic system that finds newly emerging AI/ML/DS terms in live research text, decides whether they are genuinely new and growing using independent data, and explains each one using only quoted sources.

1 · DISCOVER

Corpus: today's arXiv + 3-month sample + news + repos

Acronym definitions (Schwartz–Hearst) + title phrases

Keep if ≥3× more common than baseline

2 · VERIFY

OpenAlex: took off ≤3 years ago, no real presence 5–10 years ago

arXiv: ≥5 papers in last 30 days

Still growing (year pace or 30-day vs prior 150-day rate)

3 · EXPLAIN & KEEP

Definition: Wikipedia or the plainest verbatim abstract sentence

Usage sentence + sources + numbers

Stored permanently, grouped by month "first spotted"

1. Discovery (no hardcoded terms)

- **Recent corpus:** today's arXiv listings (cs.AI, cs.LG, cs.CL, cs.CV, stat.ML), a sample of AI/ML papers from the last 3 months (7 two-week windows × 120, via OpenAlex), news from the last 90 days, Hugging Face papers and GitHub repos. Job ads are excluded from mining (they're full of HR acronyms) and only counted later.
- **Acronyms are learned from text** with the **Schwartz & Hearst (2003)** abbreviation algorithm: find "long form (SF)" patterns and match the short form's letters backwards through the preceding words. E.g. "Cache-Augmented Generation (CAG)" → CAG; it correctly ignores "2024 (USA)".
- **Phrases:** 2–3-word title n-grams appearing in ≥4 documents.
- **Novelty ("lift"):** rate in the recent corpus ÷ rate in a **baseline** of AI/ML papers from 6–18 months ago (6 windows × 200, via OpenAlex) plus older AI-blog posts. Only terms ≥3× more common now go forward. Both corpora count the term the same way (long form *or* acronym) — an early bug made "API" look new because the baseline only counted the long form.
- **Spelling variants** with the same acronym (e.g. "RL with verifiable rewards" / "reinforcement learning with verifiable rewards") are merged, and the one the literature uses most is verified.

2. Verification (independent data)

The "took off" rule (OpenAlex publication history)

For each term, OpenAlex returns the number of scholarly works per year containing the exact phrase. **baseline** = median count of the years Y−10 … Y−5. If baseline > 30 works/yr, or > 2% of today's volume → **established** (rejected).
threshold = max(10, 5 × baseline). **Took off** = first year whose count ≥ threshold.
Accepted only if it took off in the last 3 years. **New** = took off this year or last; otherwise **Rising**.

Term (real OpenAlex data)	History	Result
software development kit	101–152 works every year from 2015 to 2026	Established → rejected

differential privacy	median 737/yr in 2016–2021	Established → rejected
world models	median 296/yr then a boom	Established → rejected
vision-language-action	2022: 1, 2023: 9, 2024: 63, 2025: 590, 2026: 2,395	Took off 2024 → Rising
flow matching	13–26 a year in 2015–2022, then 69 → 315 → 1,023 → 3,038	Took off 2024 → Rising (measured against its own past level)
on-policy self-distillation	2026: 208, nothing before	New
forward deployed engineer	8 works in total	Too little evidence → rejected

- **Active now:** ≥5 arXiv papers in the last 30 days (arXiv's own API, AI/ML categories; OpenAlex fallback).
- **Still growing:** this year's annualised pace ≥ last year, or the 30-day rate ≥ 1.2× the prior 150-day rate.
- **Tools from job posts** (e.g. LangGraph): mixed-case product names asked for by ≥3 employers, confirmed by **two registries agreeing** — a GitHub repo of that exact name with ≥1,000 stars *and* a PyPI or npm package of the same name that links back to that repo. "New" = first released under 2 years ago; otherwise the tool must be rising in job posts (share of the last 45 days ≥1.5× the older share). (Exact-name matching alone once matched "SaaS" to an unrelated repo.)

3. Explanation (quoted, never invented)

- **What it is:** the Wikipedia summary if a page whose title contains the term exists (via Wikipedia search); otherwise the best definitional sentence from recent abstracts. Sentences where the term is the subject ("X is…", "X refers to…", "X, a …") score highest; papers describing their own system ("we propose…") are penalised; a readability score prefers plainer wording.
- **Where it's used:** a sentence with an application cue ("used in", "applied to", "enables"…), excluding results sentences full of numbers and self-descriptions; an acronym glued into another name (VR-JEPA) doesn't count as the term.
- **Sources:** the OpenAlex history link, recent arXiv papers, news, repos and job posts — top 3 shown on each card.
- **Optional plain-English rewrite** by Gemini, kept only if it passes the grounding check (Section 12).

Persistence, budgets and ”never guess”

- Verified terms are stored permanently and shown instantly on launch; a background scan looks for new ones at most every 3 hours. Verified terms are re-checked every 3 days, rejected ones every 10 days; rejected ones are forgotten after 30 days.
- When rules change, nothing is wiped: terms are labelled "re-checking", re-verified first, and removed only if they fail.
- Without a key, OpenAlex gives a small daily budget shared per network IP (in testing, roughly 100 searches a day). The radar screens at most 20 new candidates per scan (plus pending re-checks) and spends at most 12 arXiv checks, supports an optional free personal key, and if OpenAlex's budget runs out it **pauses and says so** instead of guessing. arXiv calls are retried with backoff and fall back to OpenAlex.
- A quality gate refuses to mine unless the baseline has ≥500 and the recent corpus ≥300 real arXiv papers (otherwise textbook terms look "new"); if not, it pauses and retries in ~3 hours.

11 The on-device summarizer (the ML model)

Problem framing

Summaries must be ~50 words and must never contain false facts. Abstractive models (that write new text) hallucinate; small ones more so. So the task was framed as **extractive summarization**: score every sentence of the article and show the top sentences, verbatim, in original order. Faithfulness is guaranteed by construction — the model can only *choose* publisher sentences.

Model

Item	Value
Type	Binary sentence scorer: small multilayer perceptron (MLP)
Architecture	20 standardised features → 24 tanh hidden units → 1 sigmoid output
Parameters	20×24 + 24 + 24 + 1 = **529** (12 KB JSON in `assets/`)
Inference	Pure Kotlin, milliseconds per article, no ML runtime needed
Selection	Greedy by score up to ~50 words (max 60), max 3 sentences, trigram blocking to avoid repetition

The 20 features (same code in training and on the phone — no train/serve skew)

relative position · is first · is second · 1/(position+1) · length · too short · too long · title overlap · similarity to the document centroid · TextRank score · ratio of numbers · ratio of capitalised words (names) · contains a quote · starts with a dangling pronoun/connective ("He", "However") · is a question · stop-word ratio · attribution ("said", "according to") · boilerplate (newsletter, cookie, image credit…) · document length · similarity to the lead sentence.

Training

Item	Value
Data	**CNN/DailyMail 3.0.0** (CNN stories only, journalist-written highlights): 4,730 kept → 3,789 train / 489 val / 452 test. **Tech articles from the app's own feeds** with the publisher's own summary line ("dek") as reference: 194 kept → 120 / 41 / 33.
Labels	"Oracle" extraction: greedily pick 1–3 sentences maximising ROUGE-1 + ROUGE-2 F1 against the reference; those are positives. 115,957 training sentences, 9,177 positive (~8%).
Quality filters	Drop truncated/out-of-range deks, paywalled or thin pages, references not supported by the text (unigram recall <35%), deks that just copy the first sentence (36 dropped), duplicates. Deterministic hash-based splits — test articles never trained on.
Optimiser	Adam (β1 0.9, β2 0.999), lr 0.002 for 8 epochs then 0.0007, 24 epochs, batch 128, L2 1e-4, seed 42
Imbalance	Positive weight ≈5.8×; tech examples weighted 4× (domain adaptation)
Model selection	Best epoch on validation (CNN val + 2× tech val); a position prior was tuned on validation (best = 0)
Hardware	CPU, under a minute (the RTX 3050 wasn't needed)

Results (held-out test sets, ROUGE F1 × 100, 50-word budget)

Method	Tech R-1	Tech R-2	Tech R-L	CNN R-1	CNN R-2	CNN R-L
Lead (first sentences)	27.4	11.1	20.6	31.6	11.6	21.0
TextRank	24.9	7.7	17.8	29.5	9.0	19.2
sudo model	**28.4**	**11.8**	**21.1**	**33.0**	**12.6**	**22.0**
Oracle (ceiling)	38.4	21.0	29.8	45.4	24.2	32.8

Lead is famously hard to beat in news (journalists put key facts first). Honest caveat: the tech test set is small (33 articles), so treat that row as encouraging, not conclusive. More tech data accumulates with each retrain.

> **Why so small a model?**
>
> Choosing sentences is far easier than writing text; 116k examples would be memorised by a big model; it must run in milliseconds in the background on a phone; and every decision is explainable through 20 human-readable features. A fine-tuned abstractive model (e.g. Gemma 3 270M with LoRA) was considered and rejected for the default path because it would be ~500,000× larger and less faithful.

12 Where LLMs are used (Gemini) — and how they're kept honest

All LLM features are optional (the user supplies a free Gemini key) and the app works fully without them.

Feature	Prompt constraint	Safeguard
Rewritten summaries + "why it matters"	"Use ONLY the supplied text; no fact, number, name or claim not in it"; JSON output	IDs validated; length checked; labelled "AI summary"; off by default
Resume parsing (PDF sent as inline data)	"Use only what is written; leave fields empty if absent"	User reviews every field before saving
Match analysis	Honest coach; no invented requirements	Labelled "AI analysis"; local score still shown
Cover letter + answers	Only resume facts; don't claim missing skills	Editable; user submits
Radar plain-English rewrite	Only facts from the sources	**Grounding check**, else the verbatim quote is shown

The grounding check (`TermMiner.grounded`)

For every generated sentence: at least 70% of its content words (lightly stemmed) must appear in the sources, **every number** must appear verbatim, and **every capitalised name** must appear verbatim. Anything else is rejected. Unit tests prove it rejects an invented "40 times faster", an invented "invented at Microsoft" and an unsupported claim about hospitals.

Model handling: default `gemini-2.5-flash-lite` (free tier, cheap); if the model is retired (HTTP 404) the client lists available models and switches to the newest stable Flash-Lite model (or the newest Flash model); rate

limits (429) pause AI features until the next refresh with a clear message.

13 Background work, performance & memory

Refresh schedule

- A **WorkManager periodic job every 15 minutes** (Android's minimum) when online: news every run, jobs at most hourly, radar at most every 3 hours. Android may delay it in Doze; a banner (and Settings) offers to allow unrestricted battery use so it keeps running.
- Opening the app **doesn't trigger a visible refresh**: cached data shows instantly; a silent catch-up runs only if the worker hasn't refreshed in 20 minutes. The pull-to-refresh spinner appears only when the user pulls.

Three production problems and their fixes

1 · "isn't responding" (ANR)

Feed parsing, job parsing and term mining ran on the UI thread. **Fix:** all heavy work moved off the main thread; list filtering/clustering via `flowOn(Default)`; jobs list uses a light projection (it was loading ~9 MB of descriptions on every change).

2 · App killed while scrolling (OOM)

Reproduced with a 256 MB heap: radar mining used **+142 MB**. **Fix:** don't mine job ads; index only acronym-like tokens; int-array postings; trimmed job text loaded via SQL `substr`; image cache capped at 12%. Now **+77 MB with twice the data**.

3 · CPU contention

News, jobs and radar all started at launch. **Fix:** `HeavyWork`: two dedicated threads at Android *background* priority (the scheduler always prefers the UI thread) + a mutex so memory-heavy phases run one at a time.

Proof

A live test swipes the real feed while a real refresh runs: **992 swipes, median 9 ms, p95 25 ms, max 501 ms** (ANR threshold 5,000 ms). A crash recorder saves any crash so it can be copied from Settings.

Other optimisations

- Streaming JSON decode for huge job boards; cheap tag-stripping instead of an HTML parser per RSS item.
- The app root redraws only when theme/onboarding change, not on every settings write.
- Lazy-list `contentType` hints for efficient recycling; radar sources decoded in the background.
- arXiv requests are serialised at 1 per 3 s (arXiv's policy) with exponential backoff on failures.

14 Trust & verification

Rules every news item must pass

Rule	Why
Must have a publish date; not in the future; not older than 72 h	Undated items can't honestly be called "latest"
Link must belong to the feed's publisher (brand match; feed's declared site accepted)	Blocks injected / off-site links, even from a compromised feed
http(s) only; headline required	Blocks `javascript:` and junk
No commerce: promo codes, coupons, deals, ticket/event promos	Not news

The live truth audit (`LiveTruthTest`)

Runs the app's real pipelines over the internet, then independently re-checks the output:

- every headline is identical to the publisher's feed; a sample of article pages is opened to confirm the headline (og:title / <title> / h1);
- every link is on the publisher's own site; every date valid;
- every on-device summary sentence appears word-for-word in the publisher's page or feed text (publisher edits after fetch are reported separately);
- a sample of job pages is live and shows the job title (bot-blocked or JavaScript-only pages are reported "unverifiable", never "passed");
- every Trending item matches the live Hugging Face / GitHub API (still listed, counts match, repo created in the last 30 days);
- every Skills Radar term: definition verbatim at its cited source, numbers reproducible, acronym expansion used in papers, and its age confirmed by **two independent sources** (OpenAlex "took off" and arXiv volume 4–5 years ago).

Latest audit (6 Oct 2026)	Result
Links on publisher's own site	80/80
Headlines identical to publisher's feed	80/80
Headlines matching the live page	31/31 reachable
Summaries word-for-word	15/15
Job pages live with correct title	23/23 reachable
Trending items exact	45/45
Skills Radar terms	Not checked in this run: the research sources were rate-limiting, so the radar had no terms to audit. Re-run it once they're available.

What can’t be proven (say this honestly)

Whether a publisher's own reporting is true — the app uses established outlets, shows multi-outlet coverage, and never writes news itself, but it can't fact-check journalism. Some pages block automated checks. And a phrase's earliest use may have meant something else — which is why the radar measures when a term *took off*, not when it first appeared.

15 Testing strategy

Suite	Kind	What it proves
`TrustRulesTest` (14)	Unit, offline	Fake/off-site/undated/future/promo items rejected; RSS/Atom/broken-feed parsing; acronym learning; new terms outrank old; explanations verbatim; grounding check rejects invented facts; the "took off" rule on real OpenAlex numbers
`SummarizerTest` (3)	Unit, offline	Sentence splitting; summaries verbatim and 20–60 words; boilerplate never chosen; thin text falls back
`RisingTest` (9)	Unit, offline	Trending page parsing; README quotes skip captions, badges and headings; growth evidence order; list kept unless new repos make it; every-3rd-morning schedule; eligibility; AI/ML classifier
`MigrationTest` (1)	Database	A v1 database upgrades to v2 with bookmarks and tracker intact
`AppUiTest` (10)	UI (Robolectric + Compose)	Home feed, topic filter, Rising on GitHub filters + details, AI Repos tab, AI tab + radar, 3-month timeline with sources, internship filter, job detail + assisted apply + tracker, bookmarks, onboarding
`ScreenshotTest`	Visual (Roborazzi)	Renders screens (light/dark, intro, radar) to PNG for design review
`LiveSourcesTest`	Integration, live	Every feed, job source and trending source responds with valid, dated items
`LiveTruthTest`	End-to-end, live	The truth audit (Section 14), written to `testing/TRUTH_REPORT.md`
`ScrollWhileRefreshingTest`	Performance, live	Swipe latency during a real refresh
`RisingLiveTest`	Integration, live	Runs the real Rising on GitHub update and checks every repo on github.com: link, description, README quote, weekly stars
`RadarMemoryProbe`	Memory, live	Peak memory of radar mining with a 256 MB heap
`TrainSummarizerTest`	Training	Rebuilds datasets, retrains, benchmarks; writes `ml/REPORT.md`

Why Robolectric? It runs Android and Compose code on the PC's JVM, so UI tests run in seconds without an emulator (this PC didn't have the RAM for one). **Why live tests?** The app's correctness depends on external sources that change daily; only checking against the real internet proves the output is true today. Live tests are gated behind `LIVE=1` so normal runs stay fast and offline.

16 Security & privacy

- **No backend, no analytics, no account.** Nothing about the user leaves the phone except requests to the sources listed below.
- **Data sent out:** page/API requests to news sites, job APIs, arXiv, OpenAlex, Wikipedia, GitHub, PyPI, npm, Hugging Face; Google's favicon service for source icons; Gemini only if a key is set (article text, resume

and job text for the AI features the user triggers).

- **Keys** (Gemini, Adzuna, OpenAlex) are stored in app-private DataStore. Honest note: they are not additionally encrypted; `allowBackup=false` keeps them out of cloud backups.
- **Resume** is stored in app-private storage; shared only through Android's share sheet via a FileProvider when the user taps "Share".
- **Assisted apply never submits anything**; credentials are never handled by the app.
- The repository was scanned for secrets before publishing (no keys, tokens or signing keys committed).
- Release APK is signed with a personal debug key — fine for personal distribution; a Play Store release would need a proper upload key and Play App Signing.

17 Build, release & GitHub

```
gradlew assembleDebug                    # debug APK
gradlew assembleRelease                  # R8-minified release APK (~3.9 MB)
gradlew lintRelease                      # Android lint (0 errors)
gradlew :app:testDebugUnitTest           # offline unit + UI tests
set LIVE=1 && gradlew :app:testDebugUnitTest   # + live audits
set TRAIN=1 && gradlew :app:testDebugUnitTest --tests "*TrainSummarizerTest*"   # retrain
```

- Toolchain set up from scratch: JDK 21, Android SDK command-line tools (platform 35, build-tools 34), Gradle wrapper 8.11.1.
- Repository: **github.com/JiviteshKumar/sudo** (public). Releases with the APK: v1.0.0 (2 Oct) and v1.1.0 (5 Oct). The crash-during-refresh fix and the permanent radar history (commits of 6–7 Oct) are not in a release yet; that APK has only been built locally.
- Commit history was rewritten so every commit is authored with the email linked to the GitHub account, so contributions are credited to JiviteshKumar.
- Updates install over the previous version only if signed with the same key (`~/.android/debug.keystore`) — keep it backed up.
- Bundled fonts (Inter, Space Grotesk) are SIL Open Font License; license texts are included in `licenses/` .

18 Key decisions & trade-offs

Decision	Alternatives	Why this one
Extractive on-device summaries	Abstractive LLM (cloud or fine-tuned small model)	Cannot hallucinate; free; offline; fast; explainable. Cost: less fluent than rewritten text.
Tiny MLP over hand-crafted features	BERT-style sentence encoder (e.g. ONNX)	12 KB vs tens of MB; ms inference; no ML runtime; features shared by training and app.
Assisted apply	Fully automatic submission	Terms of service, CAPTCHAs, reliability, user control.
No backend	Server that pre-fetches and serves	Zero cost, zero maintenance, privacy. Cost: each phone does its own work.
Curated source list	Open web search / aggregator	Trust anchor; avoids fake news. Cost: coverage limited to chosen outlets.
OpenAlex for "took off" + arXiv for activity	arXiv only; Google Trends; LLM judgement	OpenAlex has year-by-year counts across all scholarship; arXiv shows live activity; two sources cross-check each other; no hardcoding or guessing.
Manual DI	Hilt / Koin	Small app; fewer moving parts; easy test injection.
Room + Flow as single source of truth	In-memory caches	Offline reading, instant launch, reactive UI.
Robolectric + live tests	Emulator/instrumented tests	Fast on a low-RAM PC; live tests verify truth, not just code.
Native Kotlin	Flutter / React Native	Deep Android integration (WorkManager, widget, battery, notifications); performance. Cost: Android only.

19 Bugs found and how they were fixed

These came from testing the app's output against reality — great material for "tell me about a hard bug".

Symptom	Root cause	Fix
Some summaries not found on the article page	arXiv feeds prefix abstracts with "arXiv:... Announce Type: new Abstract:"	Strip feed-specific labels; audit checks both page and feed text
Coupon posts shown as news	Wired publishes promo-code pages in its feed	Commerce filter (+ ticket/event promos)
Different arXiv papers merged as one story	Research titles share vocabulary	Papers only cluster on identical titles
"Accounting, Internal Controls" labelled internship	Regex matched "intern" inside "Internal"	Word boundaries; strong/weak/non-tech title rules

Radar showed SDK, differential privacy, world models as new	Low-volume arXiv counts; booming old fields look new by ratio	OpenAlex "took off" rule with prior-presence limit; two-source age audit
"API", "AI", "GPU" ranked as new terms	Baseline counted only long forms; recent corpus counted acronyms too	Count both forms in both corpora; 3× lift gate
Textbook terms looked new one day	arXiv throttled bulk queries → baseline fell back to blog posts only	≥500-paper baseline gate; OpenAlex sampling; retries; pause instead of guess
RLVR missed	A rare spelling was chosen to verify	Merge variants; verify the most-used spelling
"isn't responding" while scrolling	Parsing and mining on the UI thread	Background dispatchers, flowOn, light projections
App shut down during refresh	Out of memory: +142 MB radar mining on a 256 MB heap	Memory redesign (+77 MB), one-at-a-time lock, image cap
A "we/our" detector never fired	`"\b"` in a Kotlin string is a backspace, not a regex word boundary	Use `"\\b"`; scanned for similar bugs
Onboarding didn't reach Home in tests	Navigation resumed off the main thread after a DataStore write	Navigate on `Dispatchers.Main`
Lint error blocked release	An invisible byte-order-mark character in source	Replaced with `Char(0xFEFF)`

20 Limitations & roadmap

Known limitations (be upfront about these)

- Android only; not on the Play Store yet (sideloaded APK, debug-signed).
- Can't fact-check publishers; coverage limited to the curated outlets.
- Taken-down jobs can remain visible for up to 3 days; an unreachable source's jobs disappear after 3 days.
- Job match vocabulary is a fixed list of 82 skills (explainable, but not learned).
- Tech-domain summariser test set is small (33 articles); OpenAlex's anonymous budget can run out on shared mobile IPs (optional free key).
- API keys are stored without extra encryption; `versionName` still reads 1.0.0; the latest fixes aren't in a GitHub release yet.

Roadmap ideas

- Instant removal of taken-down jobs from employer boards; live check when opening a job.
- Learn the skills vocabulary from the radar and job data.
- Retrain the summariser regularly as tech data accumulates; explore a small on-device encoder.
- Android Autofill service for application forms; encrypted key storage; Play Store release with proper signing.
- Audio briefing, deadline calendar (hackathons, internship windows), iOS via Kotlin Multiplatform.

21 Interview questions — with answers

What is sudo, in one sentence?

An Android app that gives you verified tech and AI news in 50-word summaries made on the phone, a self-updating Skills Radar of genuinely new AI/ML terms, and resume-matched jobs with assisted apply.

Why did you build it?

To stay current in a fast-moving field without doom-scrolling or trusting unverified sources — and to make job hunting faster by matching roles to my real skills.

How do you make sure the app doesn't show fake or invented information?

Four layers: curated sources only; strict acceptance rules (real dates, link must be on the publisher's own site, no promos); extractive summaries that are verbatim publisher sentences; and a live truth audit that re-checks headlines, links, summaries, job pages, trending items and radar terms against the original sources. Generated text is optional, labelled and grounding-checked.

Why not just use an LLM to summarise?

LLMs can hallucinate, cost money and need a network. My use case needs ~50 faithful words, which sentence selection does perfectly. An LLM is offered as an option, constrained to the source text and labelled.

Describe your model.

An extractive sentence scorer: 20 hand-crafted features (position, length, title overlap, centroid similarity, TextRank, boilerplate flags…) into a 24-unit tanh MLP with a sigmoid output — 529 parameters. The top sentences are chosen within a 50-word budget with trigram blocking.

How did you get labels for training?

Oracle labelling: for each article I greedily selected the 1–3 sentences that maximise ROUGE-1 + ROUGE-2 against a human reference (CNN journalist highlights, or the publisher's own summary line for tech articles). Those sentences are positives.

How did you evaluate it, and is it good?

ROUGE F1 on held-out test sets against Lead, TextRank and the oracle ceiling. It beats Lead on both: 28.4 vs 27.4 ROUGE-1 on tech, 33.0 vs 31.6 on CNN. Lead is a notoriously strong baseline in news; the tech test set is small, so I describe that result as encouraging.

How did you avoid train/serve skew?

The feature code is one Kotlin file used by both the training job (run as a JVM test) and the app at inference. The trained weights are exported as JSON into the app's assets.

How does the Skills Radar decide a term is new?

Discovery: acronyms learned via the Schwartz–Hearst algorithm plus title phrases, kept if ≥3× more frequent than in papers from 6–18 months ago. Verification: OpenAlex's year-by-year publication counts must show the term took off within 3 years with almost no presence 5–10 years earlier; it must have ≥5 arXiv papers in the last 30 days and still be growing. It's then explained with a quoted source.

How do you handle a phrase that existed before with a different meaning?

I don't use "first appearance". I measure when volume surged above the phrase's own historical baseline (5× the median of years 5–10 ago). "Flow matching" had ~20 unrelated uses a year, then jumped to 315 in 2024 — so it's dated 2024.

What if your data sources are down or rate-limited?

The radar never guesses: it requires a minimum-size research corpus, retries arXiv with backoff (falling back to OpenAlex), detects OpenAlex budget exhaustion, and pauses with a visible message. Unchecked terms are neither passed nor rejected.

What was the hardest bug?

The app being killed while scrolling during a refresh. I reproduced it by running the radar's mining on real data with a 256 MB heap: +142 MB. I redesigned the mining (excluded job ads, acronym-only token index, int-array postings, SQL-trimmed text), serialised heavy jobs with a mutex on background-priority threads and capped the image cache — +77 MB with twice the data, and a test showing 992 swipes during a live refresh with a 25 ms p95.

How does background refresh work on Android?

WorkManager periodic work every 15 minutes (the minimum) with a network constraint. Doze can defer it, so the app requests a battery-optimisation exemption. Opening the app shows cached data and only catches up silently if the worker hasn't run recently.

Why Kotlin and Compose?

Native access to WorkManager, notifications, widgets and battery settings; performance for long lists; and the core logic runs unchanged in JVM tests. Compose made the animated, gradient-heavy UI straightforward.

How is the job match score computed?

Skills mentioned in the job (from an 82-skill vocabulary with aliases) are compared with the user's; score = matched ÷ min(required, 10). It's deterministic and explainable. An optional Gemini analysis adds a holistic view.

Why not auto-apply to jobs?

It violates job-site terms, breaks on CAPTCHAs and custom forms, and removes the user's review. Assisted apply gets 90% of the speed safely.

How do you keep the app responsive?

No heavy work on the UI thread; a dedicated two-thread dispatcher at background priority; a mutex so heavy phases don't overlap; flowOn for list transforms; light DB projections; capped image memory. Verified with a swipe-latency test during a live refresh.

How do you test something that depends on the live internet?

Pure logic is unit-tested offline with fixtures; UI is tested with Robolectric against a seeded in-memory database; and live, opt-in tests run the real pipelines and verify the output against the sources, reporting unverifiable cases instead of passing them.

How does ”Rising on GitHub” know a repo is rising, not just big?

It ranks by stars gained per day, never by total stars: GitHub Trending's own weekly count, else the gain sudo measured between two of its checks, else a new repo's average since launch. Each card names its source. The list is renewed every 3rd morning only if a different set of repos makes the top 15.

What would you do with more time?

Instant removal of taken-down jobs, a learned skills vocabulary, a Play Store release with proper signing, encrypted key storage, more tech training data, and an Autofill service for job forms.

Is there anything hardcoded?

The list of trusted outlets and job boards (deliberately — it's the trust anchor), the 82-skill vocabulary for match scores, and thresholds (e.g. 3× lift, 3-year window). Terms, trends and explanations are all derived from live data.

22 Glossary

Term	Meaning
ANR	"Application Not Responding" — Android's dialog when the UI thread is blocked ~5 s.
arXiv	Open repository of research preprints; its API gives counts and abstracts.
OpenAlex	Open catalogue of hundreds of millions of scholarly works; used for year-by-year publication counts and paper samples.
Extractive summarization	Summarising by selecting existing sentences (vs abstractive: writing new ones).
ROUGE-1 / -2 / -L	Overlap of words / word pairs / longest common subsequence between a summary and a reference.
Oracle	The best possible extractive selection given the reference — an upper bound and a labelling method.
TextRank	Graph-based ranking of sentences by similarity (PageRank on sentences).
Lead baseline	Taking the first sentences of an article.
Schwartz–Hearst	Algorithm that finds abbreviation definitions like "long form (SF)" in text.
Lift	How many times more frequent a term is now than in a baseline.
Grounding check	Verifying generated text only contains facts present in its sources.
Train/serve skew	Differences between how features are computed in training vs production.
WorkManager	Android library for deferrable, guaranteed background work.
Doze	Android's power-saving mode that defers background work.
Room / DAO	Android's SQLite layer / the interface defining its queries.
Compose / recomposition	Declarative UI toolkit / re-running UI functions when state changes.
StateFlow / flowOn	Observable state stream / moving a stream's work to another thread pool.
R8	Android's code shrinker and optimiser for release builds.
Robolectric	Runs Android code on the JVM for fast tests.
Custom Tabs	In-app Chrome tabs that share the browser's logins.
Rate limit / 429	A server's cap on requests; HTTP 429 means "Too Many Requests".

Generated from the sudo codebase (commit c90372e and later), `ml/REPORT.md` and `testing/TRUTH_REPORT.md`. Built with Claude Code.